

SESSION 41 • NLP CLASS

Agentic AI

AI Agents • Tool Calling • Planning • Reasoning • Multi-Step Workflows



Hands-on: Build an AI Agent

1 Hour | College Lecture | Basics to Advanced

Session Agenda



What is an AI Agent?

8 min

Definition, examples & how agents differ from plain LLMs



Planning

9 min

Breaking a goal into ordered sub-tasks



Multi-Step Workflows

9 min

Chaining tools & steps to finish real tasks



Tool Calling

10 min

Letting models use calculators, search, APIs & code



Reasoning — ReAct

10 min

Thought → Action → Observation loop



Hands-on: Build an Agent

14 min

Live walkthrough with code

From LLMs to Agents

A plain LLM answers from what it already knows, in one shot. An agent can look things up, take actions, and keep going until the goal is done.



Plain LLM Call

- Single input → single output
- No memory between turns
- Cannot fetch live data
- Cannot take real actions
- e.g. "Explain backpropagation"



AI Agent

- Plans multiple steps toward a goal
- Calls tools: search, calculator, APIs
- Observes results and adapts
- Keeps looping until goal is met
- "Book me the cheapest flight Friday"

CORE CONCEPT

What is an AI Agent?



Definition: An AI agent = an LLM given the ability to perceive a situation, plan steps, take actions using tools, and observe results — repeating this loop autonomously until a goal is achieved.

Worked Example

1

User says

"Find me 3 recent research papers on RAG and summarize them."

2

Agent plans

Search papers → Read abstracts →
Summarize → Reply

3

Agent acts

Calls a web-search tool, then a PDF-
reading tool

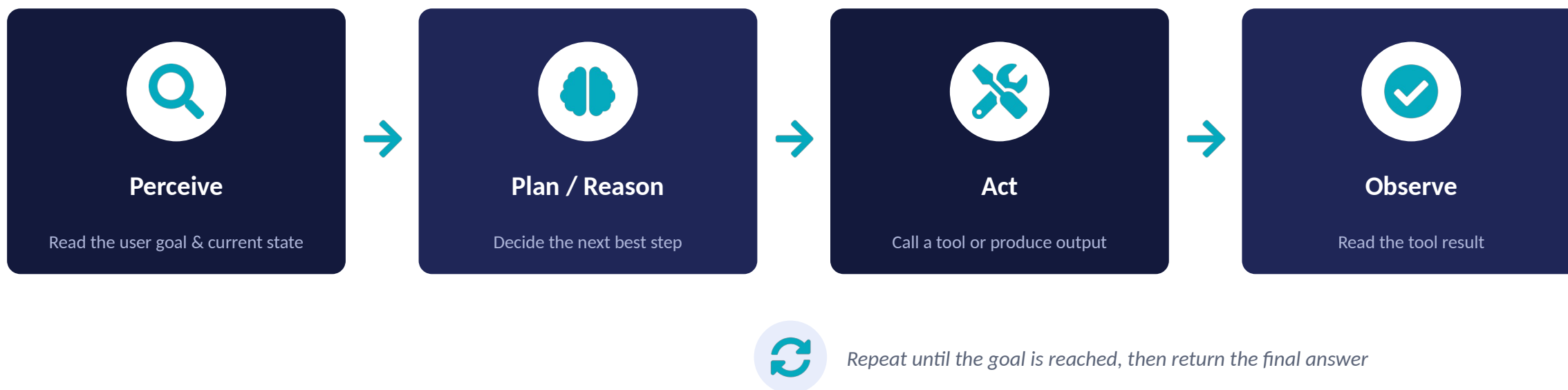
4

Agent delivers

Returns a structured 3-paper summary,
unaided after the prompt

Anatomy of an Agent: The Loop

Every agent — simple or complex — runs the same core loop until the task is done.



COMPARISON

Agent vs Chatbot vs Fixed Workflow

	Chatbot	Fixed Workflow	AI Agent
Decides next step itself?	No	No (hardcoded)	Yes
Uses external tools?	Rarely	Yes (fixed order)	Yes (chosen dynamically)
Adapts mid-task?	No	No	Yes
Best for	Q&A, FAQs	Predictable pipelines	Open-ended, multi-step goals

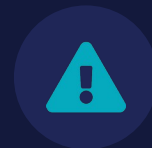
Takeaway: agents earn their complexity when the task needs judgment + tools + multiple steps.

Tool Calling — What & Why



What is it?

Giving an LLM access to external functions — a calculator, a web search, a database query, or any API — that it can call mid-conversation to get real results.



Why it's needed

- LLMs only know what they were trained on — no live data
- LLMs are unreliable at precise math
- LLMs cannot act in the real world on their own
- Tools turn "knowing about" into "actually doing"

Tool Calling — How It Works



Tool schema the model sees

```
{
  "name": "get_weather",
  "description": "Get current weather for a city",
  "parameters": {
    "location": "string", "date": "string" }
}
```

Tool Calling — Example Walkthrough



User

"Will it rain in Chennai tomorrow?"



Model

Decides to call `get_weather(location="Chennai", date="tomorrow")`



Tool runs

Weather API returns: `{ "rain_chance": "78%" }`



Model replies

"Yes, there is a high chance of rain (78%) in Chennai tomorrow."

Planning — Concept



Definition: Planning is breaking a complex goal into an ordered set of smaller, achievable sub-tasks — before or while acting — instead of trying to solve everything in one giant step.



Complex tasks need multiple steps

An LLM answering in one shot often skips steps or makes errors



Plans can be revised

If a step fails or returns unexpected results, the agent re-plans

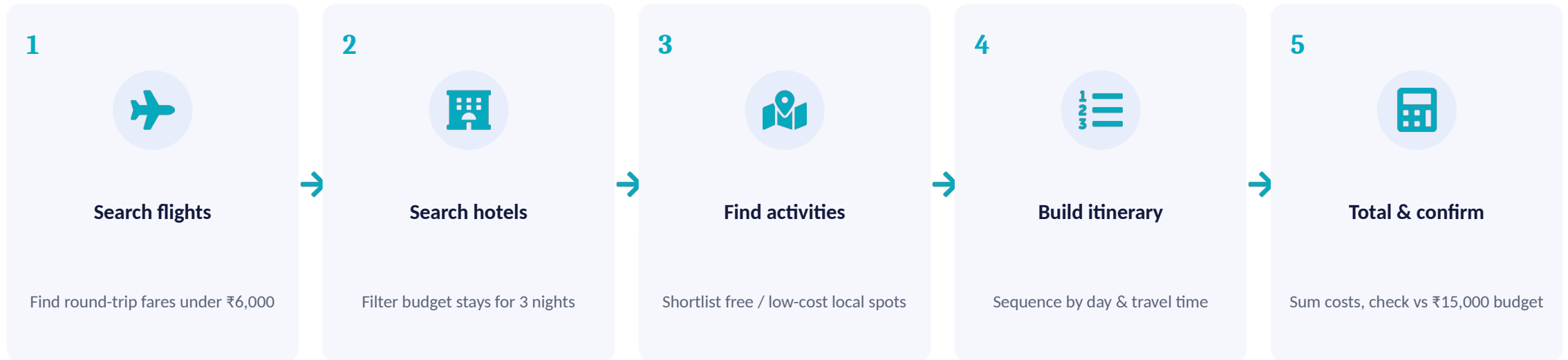


Keeps the agent on track

A plan acts as a checklist toward the original goal

Planning — Task Decomposition Example

Goal: "Plan a 3-day trip to Goa within ₹15,000."



Reasoning — The ReAct Framework



ReAct = Reasoning + Acting, interleaved. The model thinks out loud, takes an action, observes the result, then thinks again — instead of jumping straight to an answer.



Thought

What do I need to do next, and why?



Action

Call a tool / take a concrete step



Observation

Read what came back



Repeats until the model is confident enough to give a final answer

ReAct — Example Walkthrough

Q: "What is the population of the capital of France, divided by 1,000?"



Thought

I need the capital of France first.



Action

`search("capital of France")`



Observation

Paris



Thought

Now I need Paris's population.



Action

`search("population of Paris")`



Observation

~2,100,000



Thought + Action

`calculator(2100000 / 1000)`



Final Answer

2,100

Each step is short and verifiable — this is what makes ReAct easier to debug than a single black-box answer.

Multi-Step Workflows



Concept: Real tasks rarely end after one tool call. Agents chain several reasoning + tool-call loops together, carrying state forward, to complete an entire job.

Example — Travel Booking Agent



State (dates, budget, preferences) carries across every step — that's what makes it a workflow, not isolated calls.

Popular Agent Frameworks



LangChain / LangGraph

Graph-based chains, tools & memory for building agents



CrewAI

Orchestrates teams of role-based cooperating agents



AutoGPT

Early autonomous agent that plans & self-prompts toward a goal



Microsoft AutoGen

Multi-agent conversation framework for complex tasks



Function/Tool Calling APIs

Native tool-use in OpenAI, Anthropic & other model APIs



Claude Code / Code Agents

Coding agents that plan, edit files & run commands

Challenges & Limitations



Hallucinated tool calls

Model may invent arguments or call tools that don't exist



Infinite / runaway loops

Agent keeps acting without converging on an answer



Cost & latency

Every step is an extra LLM call — slower and pricier



Error handling

Tool failures, bad data & timeouts must be handled gracefully



Security risks

Prompt injection via tool results or untrusted content



Hard to evaluate

Multi-step behaviour is harder to test than single answers

Hands-on: Build a Simple AI Agent



Goal: build an agent that can answer math questions and check the weather, by calling two tools.

1



Define tools

calculator(expression)
get_weather(city, date)



2



Send query + tools

User question and tool schemas go to the
model together



3



Execute tool call

Model picks a tool; your code runs the real
function



4



Return final answer

Result is sent back; model writes the answer
for the user

Hands-on: Code Outline



agent.py — minimal tool-calling loop

```
tools = [calculator_schema, get_weather_schema]
messages = [{"role": "user", "content": user_query}]

response = model.create(messages=messages, tools=tools)

if response.tool_call:
    name, args = response.tool_call.name, response.tool_call.arguments
    result = run_tool(name, args)          # your real function
    messages.append(tool_result(name, result))
    response = model.create(messages=messages, tools=tools)

print(response.final_text)                # answer for the user
```

Key Takeaways



An agent = LLM + tools + planning + reasoning, run in a loop until a goal is met



Tool calling lets a model act on the real world — search, compute, query, execute



Planning decomposes big goals into small, ordered, checkable steps



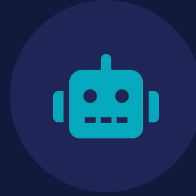
ReAct interleaves Thought → Action → Observation for traceable reasoning



Real agents chain multiple steps together, carrying state across the task



Cost, loops, and reliability are the open challenges to manage



Thank You

Questions & Discussion